

Lab session 1: Monte Carlo Simulation

Overview: Monte Carlo Simulation

A Monte Carlo study uses repeated sampling and can sometimes be used to obtain the statistical properties of an estimator $\hat{\theta}$ of a parameter θ , for example, $E[\hat{\theta}]$ or $Var(\hat{\theta})$. The parameter characterizes some population distribution. To proceed, one chooses a specific value of the parameter for the population distribution, say θ_0 . Then one draws a random sample of size N from this specific population distribution and constructs the estimates $\hat{\theta}^{(m)}$ for $m = 1, \dots, M$, all estimates of θ_0 . The sample average and the sample variance of these M estimates then are estimates of $E[\hat{\theta}]$ and $Var(\hat{\theta})$, respectively. A thorough Monte Carlo study varies the values of θ_0 , N and M . This hopefully gives one a representative view of the landscape describing the statistical properties of the estimator.

What we do in this lab session

In this note, we will perform a Monte Carlo simulation to study the properties of the sample mean. Please go over this note before we start Notes 02.

A typical Monte Carlo simulation involves generating (pseudo) random numbers. We fix the seed of the random number generator so that results below are reproducible.

```
rm(list = ls()); # refresh memory

my_random_seed = 1;          # seed for the random number generator
set.seed(my_random_seed);    # setting the seed of the random number generator
```

Building our own population

Suppose we have a population model as follows:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon$$

Because we fully control this environment, we can choose specifics of the population. In particular, we let

$$\beta_0 = 1, \beta_1 = 1, \beta_2 = 0.2$$

and

$$\varepsilon \sim t_{10}$$

where t_{10} refers to a t -distribution with degrees of freedom 10. Lastly, we assume that x follows a normal distribution:

$$x \sim N(1, 4)$$

The population object of interest

In this note, we are interested in the population mean of y . That is, our population object of interest is $E[y]$. Note that

$$E[y] = \beta_0 + \beta_1 E[x] + \beta_2 E[x^2] + E[\varepsilon]$$

Note that

$$E[x] = 1, E[x^2] = Var(x) + E[x]^2 = 5, E[\varepsilon] = 0$$

In turn,

$$E[y] = \beta_0 + \beta_1 \times 1 + \beta_2 \times 5 = 3$$

We will sometimes denote θ_0 as the population object of interest, which is $E[y]$ in this example.

Generating a data set from the population

Now, let us randomly sample y and x from the population N times. Collect them in vectors

$$Y_N^{(1)} = [y_1, y_2, \dots, y_N]$$

and

$$X_N^{(1)} = [x_1, x_2, \dots, x_N]$$

$Y_N^{(m)}$ refers to a vector of length N randomly sampled from the population of y . The superscript denotes the m -th data set. It will be clear later why we may want to have multiple data sets.

Let us write a function that generates one observation pair (y_i, x_i) .

```
generate_data = function(N = 20){  
  
  # model parameters  
  beta0 = 1;  
  beta1 = 1;  
  beta2 = 0.2;  
  
  # simulating data  
  mat_data = matrix(NA, N, 2);          # N by 2 matrix to store (y,x) by row  
  for (i in 1:N){  
    e = rt(1, df=10);                  # e ~ t distribution with degrees of freedom 10  
    x = rnorm(1, mean=1, sd = sqrt(4)); # x~N(1,4)  
    y = beta0 + beta1*x + beta2*x^2 + e; # this is our population model  
    mat_data[i,] = c(y,x);  
  }  
  colnames(mat_data) = c("y", "x"); # put column names  
  return(mat_data);  
}
```

Now, let us try to generate a data set of length 20.

```
# generating data (randomly sample) from the population  
N = 20; # length of the data  
data1 = generate_data(N); # generate_data() is a function defined in the previous code chunk
```

Sample mean

Suppose we give this data set to a researcher and ask what her guess is about the population mean of y . A sample mean is a reasonable estimator for the population mean.

$$\hat{\theta} = \frac{1}{N} \sum_{i=1}^N y_i$$

Let us calculate the sample mean based on the data set we just generated.

```
# Sample mean of y  
theta_hat = mean(data1[,1])
```

Sampling uncertainty

As we can see, $\hat{\theta}$ is close to θ_0 . However, they are not exactly the same. This is because we built our estimator based on the data set randomly sampled from the population. Quantifying this type of uncertainty (sometimes called sampling uncertainty) is an important task in econometric analysis.

In other words, we would like to know what could have happened to our estimate if we observed a different data set randomly sampled from the population.

One way to characterize such sampling uncertainty is that we (1) generate several data sets from the same population; (2) compute an estimate for each data set; (3) analyze how variable these estimates are.

Let us do it. We will generate M data sets of length N . Then, we compute the sample mean for each data set. We will end up with M sample means. Our measure of the sampling uncertainty is the variance of these sample means.

First, let us write a function that generates M data sets and calculates the sample mean of each data set.

```
simulation_sample_mean = function(N=20, M=1000){  
  # A function that generates M data sets (length of N), computes sample mean of each data set  
  # N: the length of each data set  
  # M: the number of simulations (that is, the number of data sets)  
  mat_theta_hat = matrix(NA, M, 1); # a placeholder to collect M sample means  
  for (m in 1:M){  
    dataset = generate_data(N); # generate a data set (y,x) of length N  
    theta_hat = mean(dataset[,1]); # compute sample mean of y  
    mat_theta_hat[m,] = theta_hat; # store m-th sample mean  
  }  
  return(mat_theta_hat);  
}
```

We use the variance of the sample mean as a measure of the sampling uncertainty.

```
print("Variance of the sample mean as a measure of sampling uncertainty ...")  
  
## [1] "Variance of the sample mean as a measure of sampling uncertainty ..."  
N = 20; # the length of each data set  
M = 1000; # the number of simulations (that is, the number of data sets)  
mat_theta_hat = simulation_sample_mean(N, M);  
print(var(mat_theta_hat)); # print out variance of sample means  
  
##           [,1]  
## [1,] 0.4988923
```

Thought experiment 1

As N gets larger, we can expect that this sampling uncertainty gets smaller. That is, we will have a more accurate estimate if we have more data observations. Let us examine this claim by computing the sampling uncertainty for different values of N . We try $N = 20, 100, 500$.

```
mat_theta_hat_20 = simulation_sample_mean(N=20, M);  
mat_theta_hat_100 = simulation_sample_mean(N=100, M);  
mat_theta_hat_500 = simulation_sample_mean(N=500, M);
```

We confirm that the sampling uncertainty gets smaller as we increase N . Roughly speaking it decreases by the factor of 5 when we increase N by the factor of 5.

```
print(var(mat_theta_hat_20)); # print out variance of sample means
```

```
##          [,1]
## [1,] 0.4851432
print(var(mat_theta_hat_100)); # print out variance of sample means
```

```
##          [,1]
## [1,] 0.09813125
print(var(mat_theta_hat_500)); # print out variance of sample means
```

```
##          [,1]
## [1,] 0.01991697
```

Thought experiment 2

Now, let us analyze the sampling uncertainty of the $\sqrt{N}\hat{\theta}$. Note that

$$\text{Var}(\sqrt{N}\hat{\theta}) = N\text{Var}(\hat{\theta})$$

As you will see below, the sampling uncertainty of $\sqrt{N}\hat{\theta}$ does not decrease when we increase N . In fact, they are roughly the same.

```
print(var(sqrt(20)*mat_theta_hat_20)); # print out variance of sample means
```

```
##          [,1]
## [1,] 9.702865
```

```
print(var(sqrt(100)*mat_theta_hat_100)); # print out variance of sample means
```

```
##          [,1]
## [1,] 9.813125
```

```
print(var(sqrt(500)*mat_theta_hat_500)); # print out variance of sample means
```

```
##          [,1]
## [1,] 9.958484
```

Quiz

Do the same analysis as above but with a different population model. More specifically, modify the assumption about the distribution of ε as follows:

$$\varepsilon = v - 20$$

where $v \sim \chi^2(20)$ (i.e., chi-squared distribution with 20 degrees of freedom).

- Do you arrive at the same conclusion with a new population model?
- Plot the histogram of $\sqrt{N}\hat{\theta}$ when $N = 500$ and compare it to the one based on the original population model. Do they look similar?